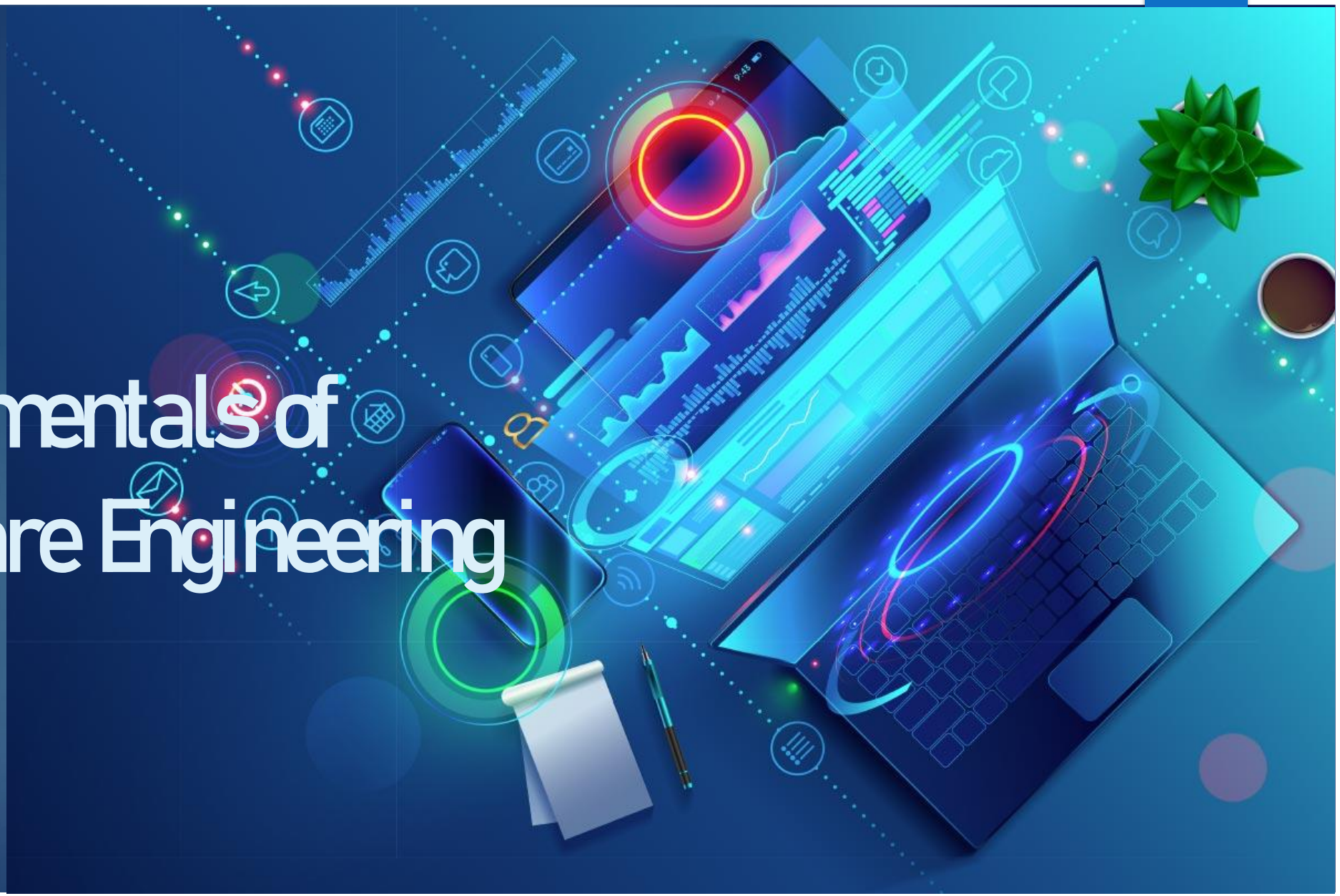
IS1204 Fundamentals of Software Engineering

H N D THILINI



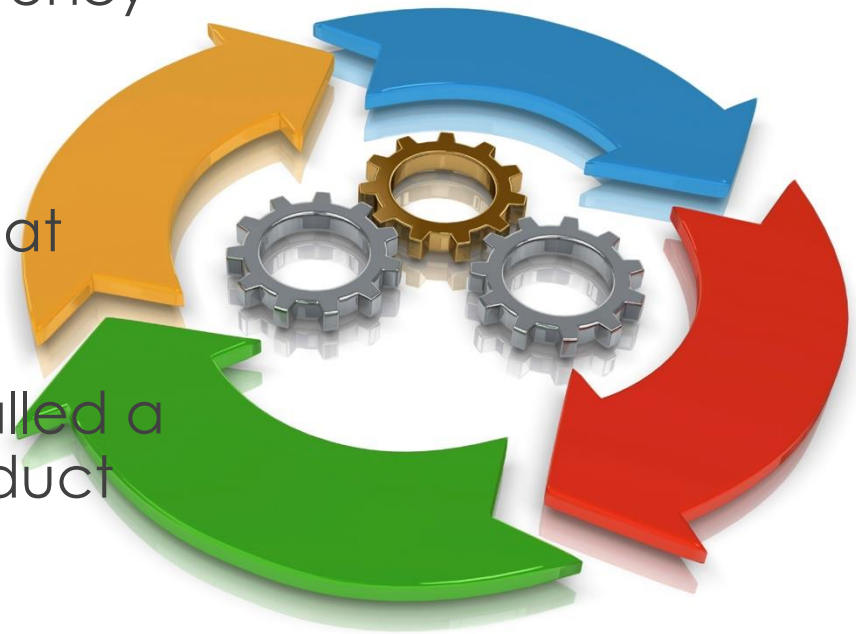
Software Process Models

Learning Outcomes

- Appreciate the need for a defined software process
- Be able to describe in detail the main software process models
- Be able to compare software process models and choose between them

What is a Software Process

- A set of ordered tasks involving activities, constraints and resources that produce a software system
- A process is important because it imposes consistency and structure on a set of activities
- It guides our actions by allowing us to examine, understand, control and improve the activities that comprise the process
- The process of building a product is sometime called a *lifecycle* because it describes the life of that product from conception through to its implementation, delivery, use and maintenance

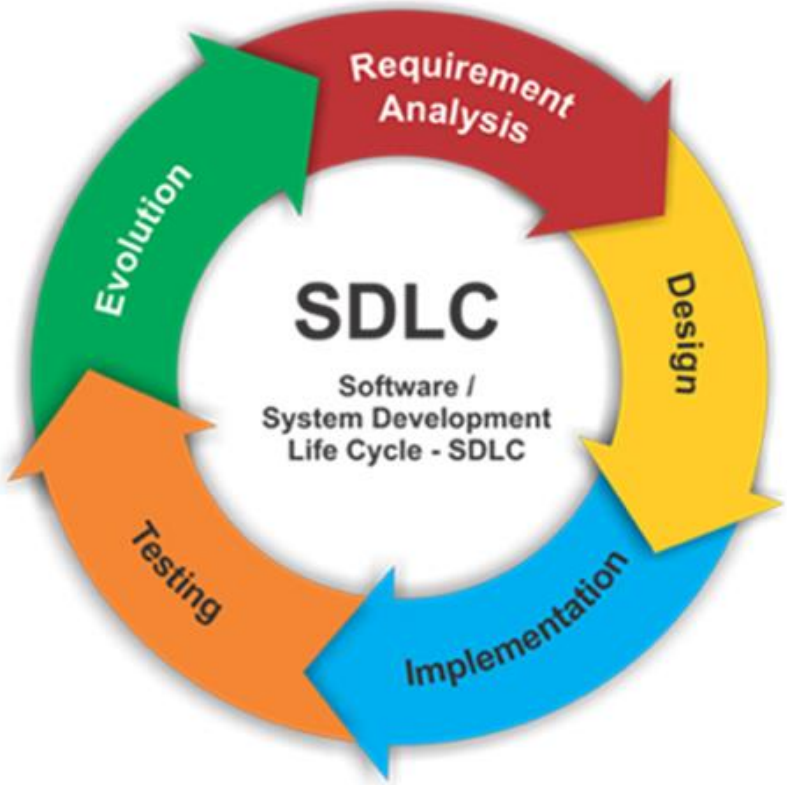


Software Process Models

- By modeling the process:
  - forms a common understanding of the activities, resources and constraints involved in software development
  - helps the team find inconsistencies, redundancies and commissions in the process
  - process becomes more effective
  - the model reflects the goals of development and shows explicitly how the product characteristics are to be achieved
  - model helps people to understand these differences in deferent developments

Software Process Models

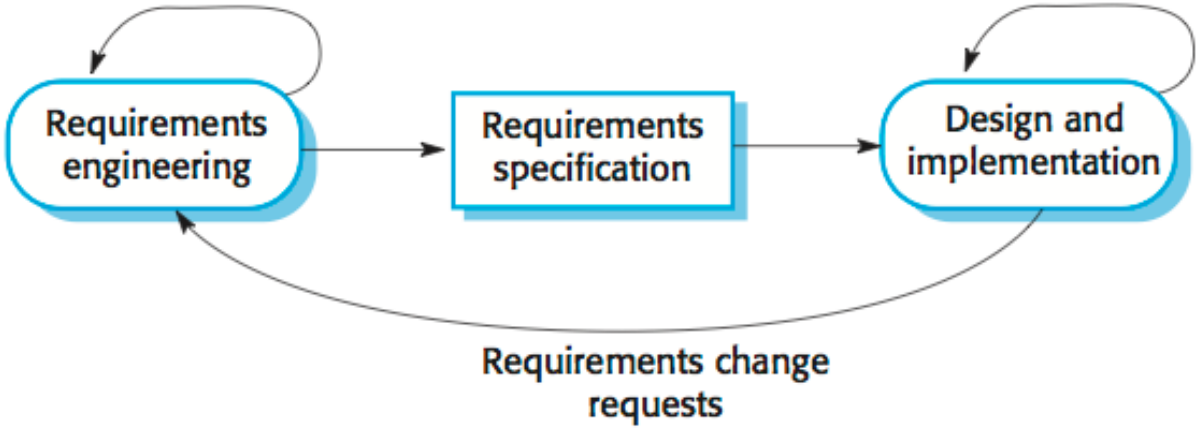
- Many different software processes but all involve:
  - **Specification** – defining what the system should do;
  - **Design and implementation** – defining the organization of the system and implementing the system;
  - **Validation** – checking that it does what the customer wants;
  - **Evolution** – changing the system in response to changing customer needs.



Software Process Models

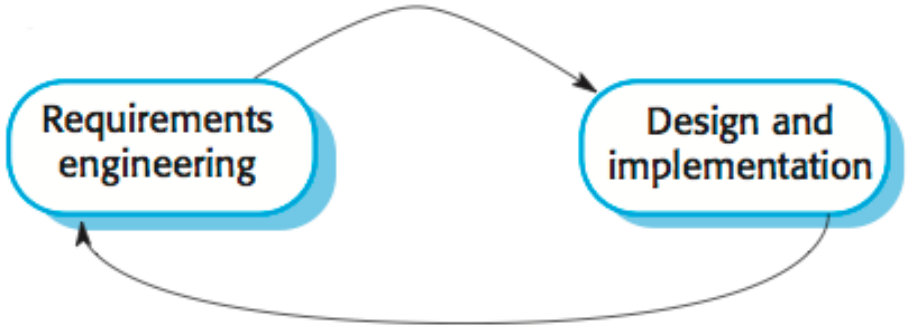
- **Plan-driven processes**

all of the process activities are planned in advance and progress is measured against this plan.



- **Agile processes**

planning is incremental and it is easier to change the process to reflect changing customer requirements.

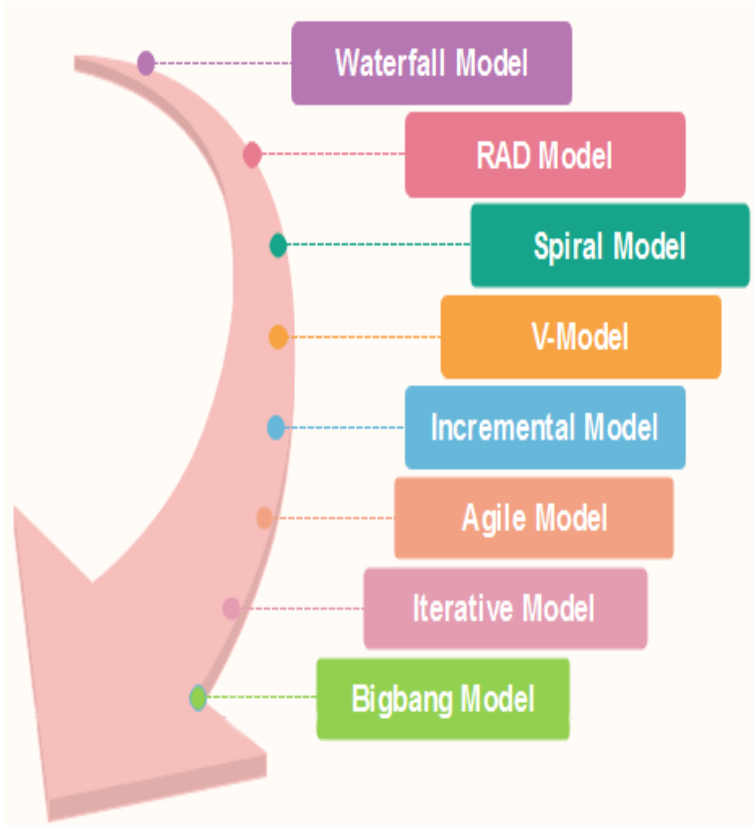


- In practice, most practical processes include elements of both plan-driven and agile approaches.

Software Process Models

- Waterfall Model
- Incremental Development
- Reused-Oriented Software Engineering
- Iterative Development
- Prototyping Models
- The Spiral Model
- Rapid Application Development

- Scrum Model
- Extreme Programming (XP)
- Dynamic Development System model
- Kanban
- Feature Driven Development



Plan-Driven Processes

1. Waterfall Model
2. Incremental Development
3. Reused-Oriented Software Engineering
4. Iterative Development
5. Prototyping Models
6. The Spiral Model
7. Rapid Application Development

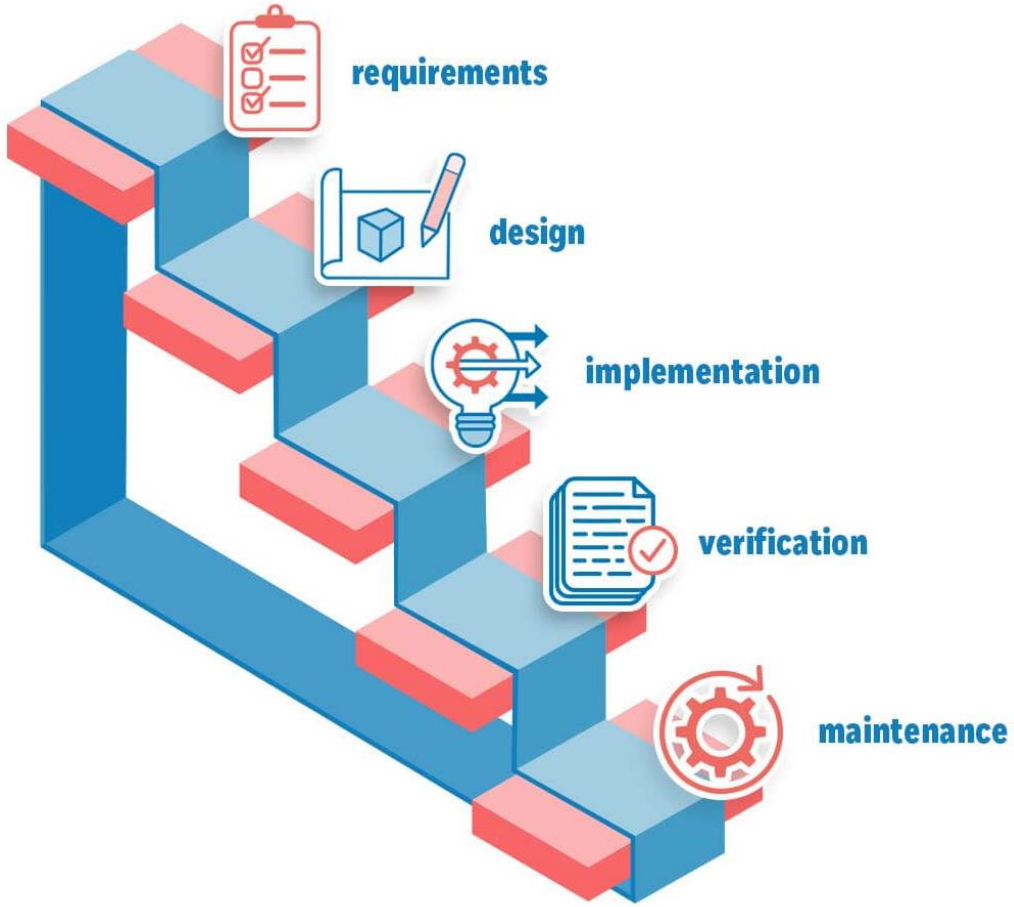
- What is it?
- Features / Properties / Stages
- What type of projects is it suitable for
- Advantages/pros and disadvantages/cons

Waterfall Model

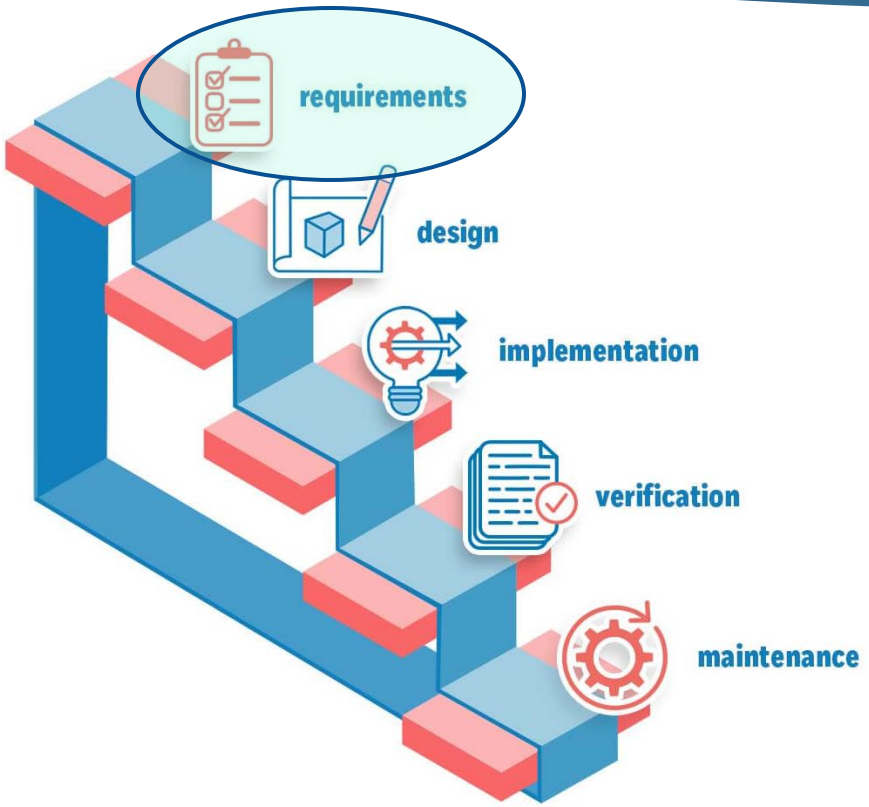
Waterfall Model

- The first published model of the software development process
- Is an example of a plan-driven process.
- In principle at least, you plan and schedule all of the process activities before starting software development.
- Separate and distinct phases of specification and development
- A linear sequential model

Waterfall Model

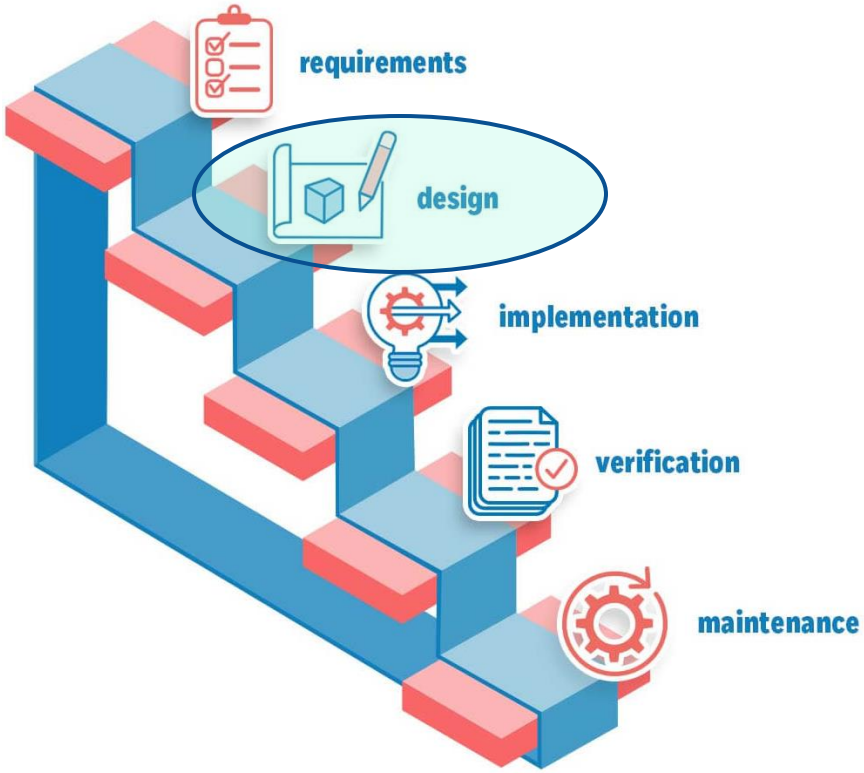


Waterfall Model



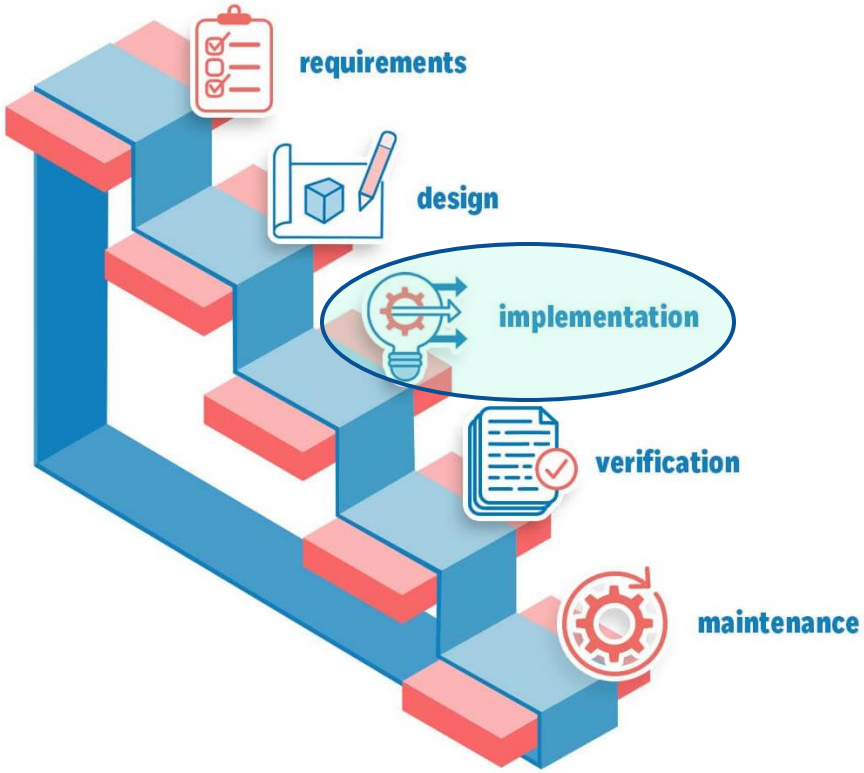
- Requirements analysis and definition
  - The system's services, constraints, and goals are established by consultation with system users.
  - Need to define those in detail and serve as a system specification.

Waterfall Model



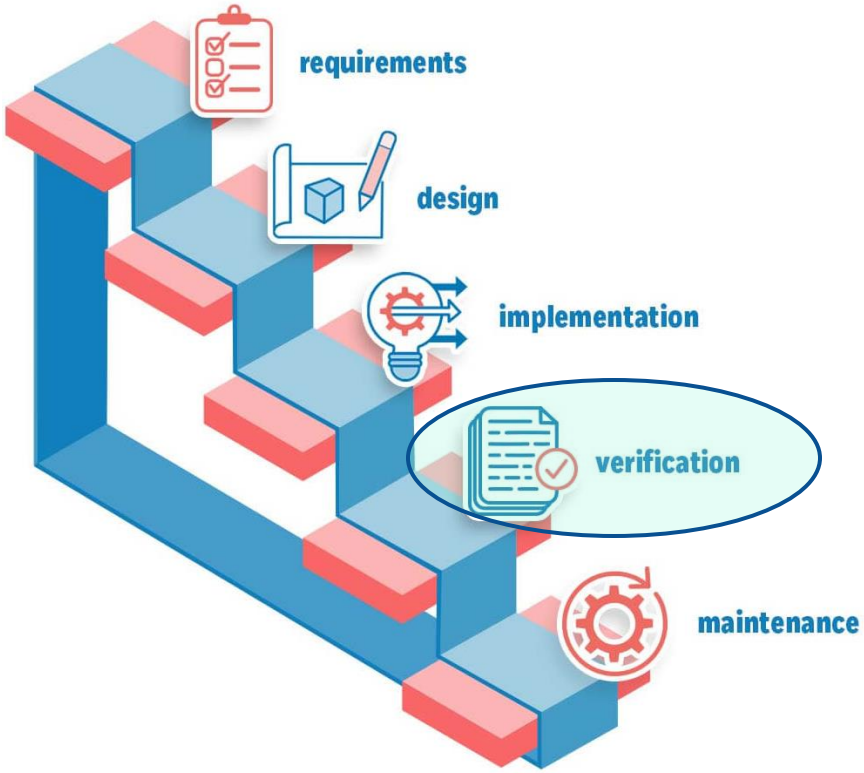
- System and software design
  - Allocates the requirements to either hardware or software systems
  - It establishes an overall system architecture.
  - Software design involves identifying and describing the fundamental software system abstractions and their relationships.

Waterfall Model



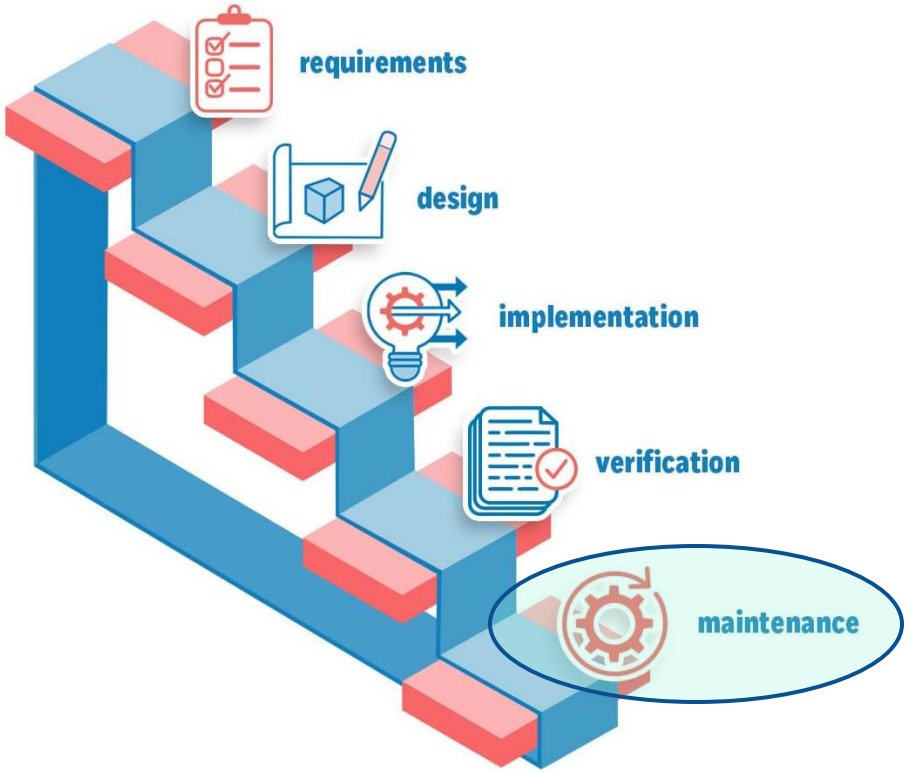
- Implementation and unit testing
  - During this stage, the software design is realized as a set of programs or program units.
  - Unit testing involves verifying that each unit meets its specification.

Waterfall Model



- Integration and system testing
  - The individual program units or programs are integrated and tested as a complete system to ensure that the software requirements have been met.
  - After testing, the software system is delivered to the customer.

Waterfall Model



- Operation and maintenance
  - Normally, this is the longest life-cycle phase.
  - The system is installed and put into practical use.
  - Maintenance involves
    - correcting errors that were not discovered in earlier stages of the life cycle
    - improving the implementation of system units
    - enhancing the system's services as new requirements are discovered.

Waterfall Model

- Waterfall model can be used when
  - Requirements are not changing frequently
  - Application is not complicated and big
  - Project is short
  - Requirement is clear
  - Environment is stable
  - Technology and tools used are not dynamic and is stable
  - Resources are available and trained

Waterfall Model

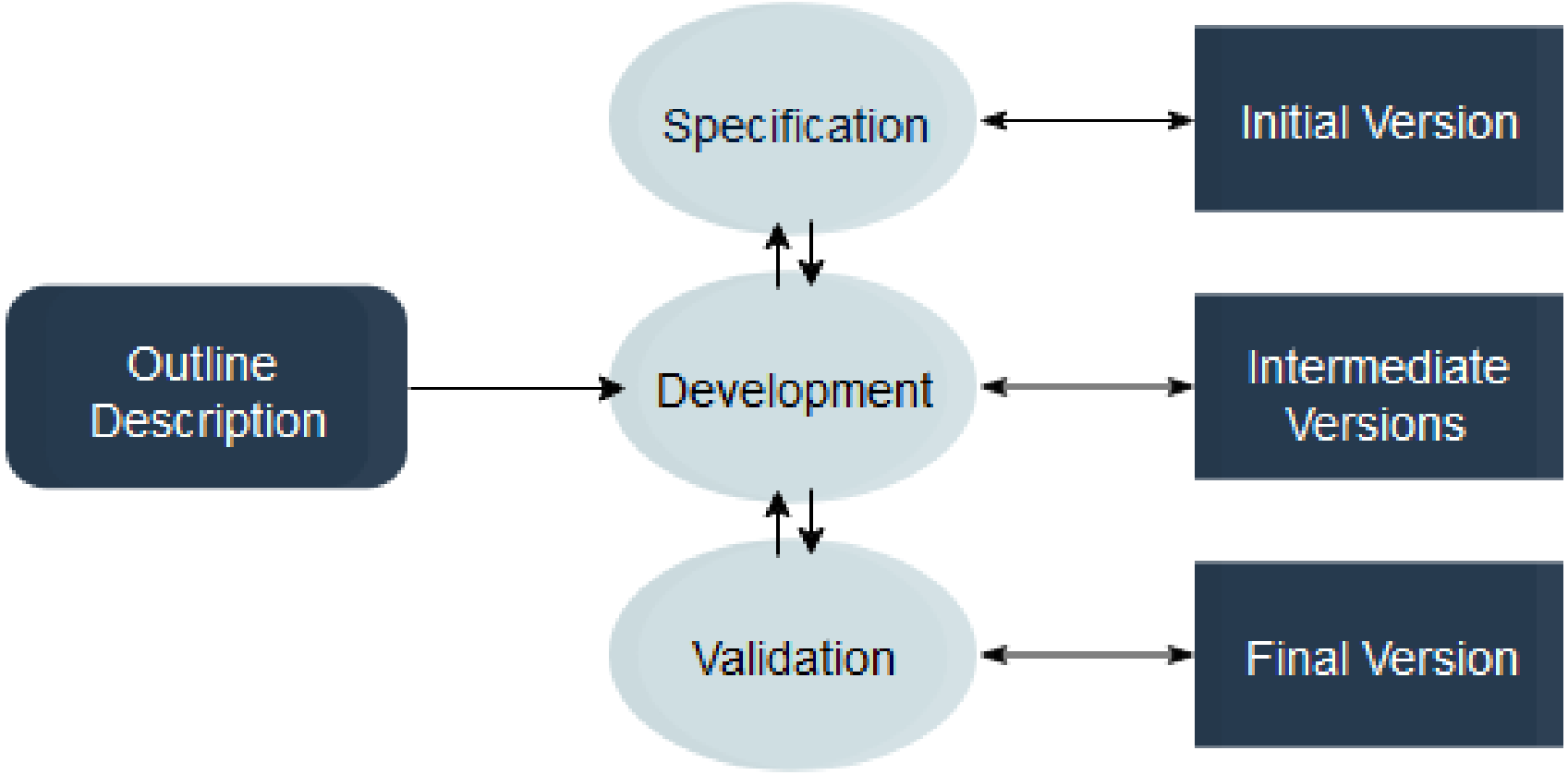
Advantages	Disadvantages
Simple and easy to understand and use.	Once an application is in the testing stage, it is very difficult to go back
Easy to manage due to the rigidity of the model	No working software is produced until late during the life cycle
Phases do not overlap	High amounts of risk and uncertainty
Works well for smaller projects where requirements are clearly defined and very well understood.	Not a good model for complex and object-oriented projects
	Poor model for long and ongoing projects
	Not suitable for the projects where requirements are at a moderate to high risk of changing.

Incremental Development

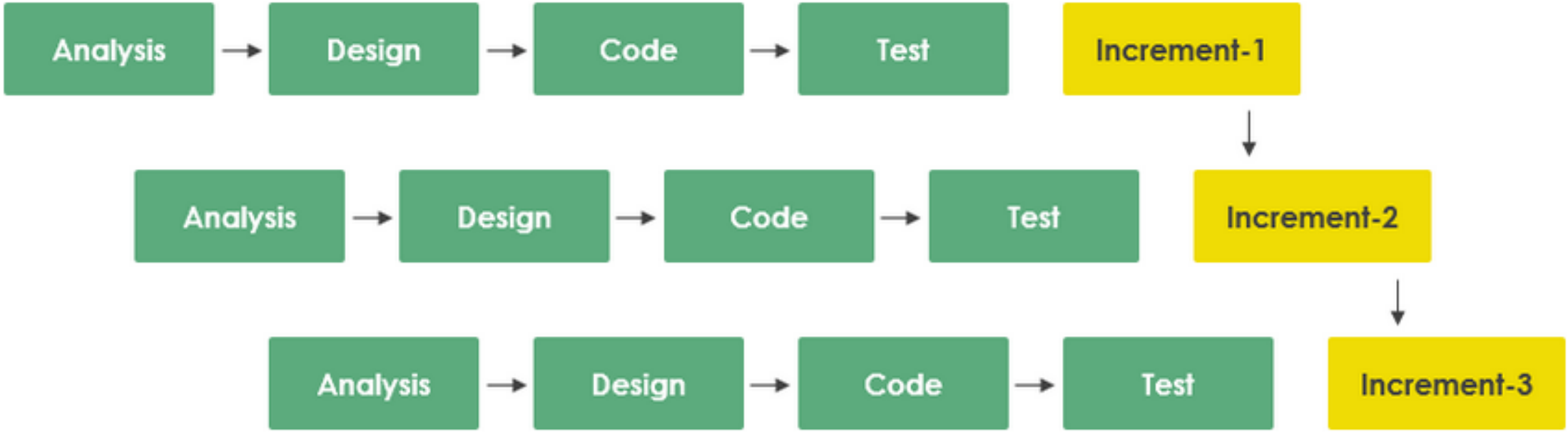
Incremental Development

- Based on the idea of developing an initial implementation, getting feedback from users and evolving the software through several versions until the required system has been developed.
- System development is broken down into many mini development projects
- Partial systems are successively built to produce a final total system
- Highest priority requirement is tackled first

Incremental Development



Incremental Development



Incremental Development

- Benefits
  - The software will be generated quickly during the software life cycle
  - It is flexible and less expensive to change requirements and scope
  - Through out the development stages, changes can be done
  - This model is less costly compared to others
  - A customer can respond to each building
  - Errors are easy to be identified

Incremental Development

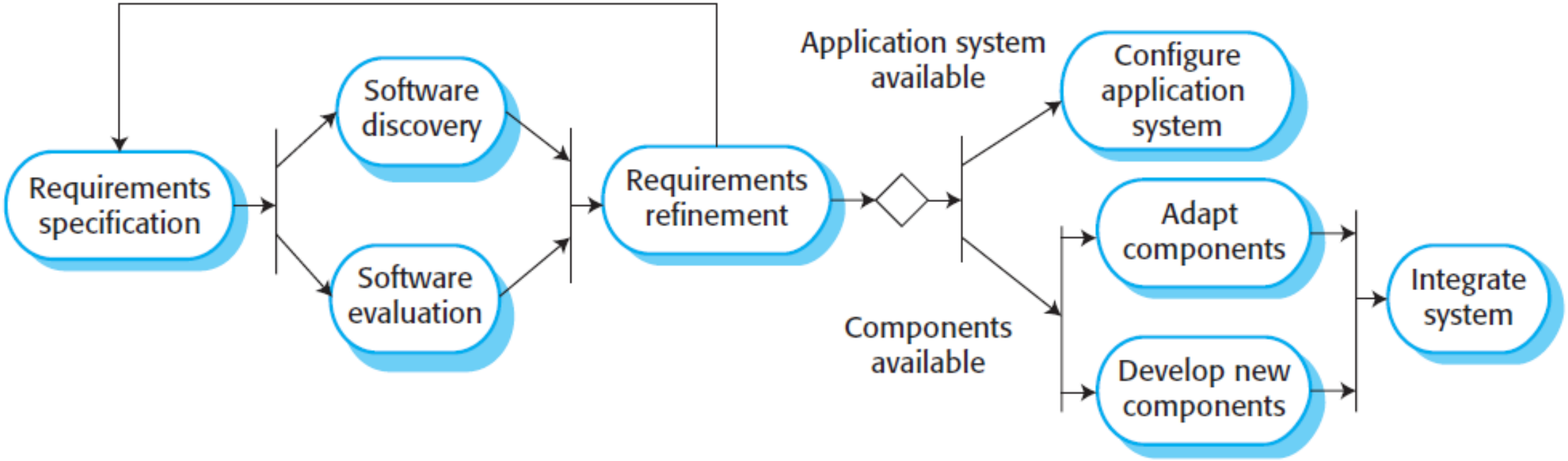
- Problems:
  - The process is not visible.
    - Managers need regular deliverables to measure progress. If systems are developed quickly, it is not cost-effective to produce documents that reflect every version of the system.
  - System structure tends to degrade as new increments are added.
    - Unless time and money is spent on refactoring to improve the software, regular change tends to corrupt its structure. Incorporating further software changes becomes increasingly difficult and costly.

Reuse-Oriented Software Engineering (*Integration and Configuration*)

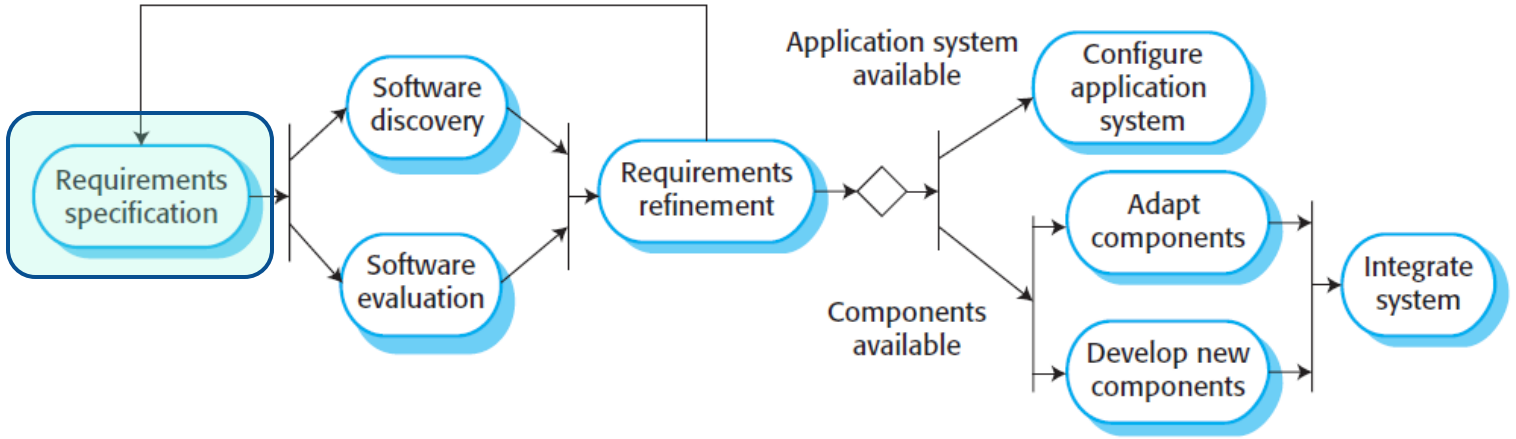
Reuse Oriented Software Engineering

- Also known as reuse-oriented development (ROD)
- Is a method of software development in which a program is refined by producing a sequence of prototypes called models,
- Every system is derived from the previous one with constant series of defined rules.

Reuse Oriented Software Engineering

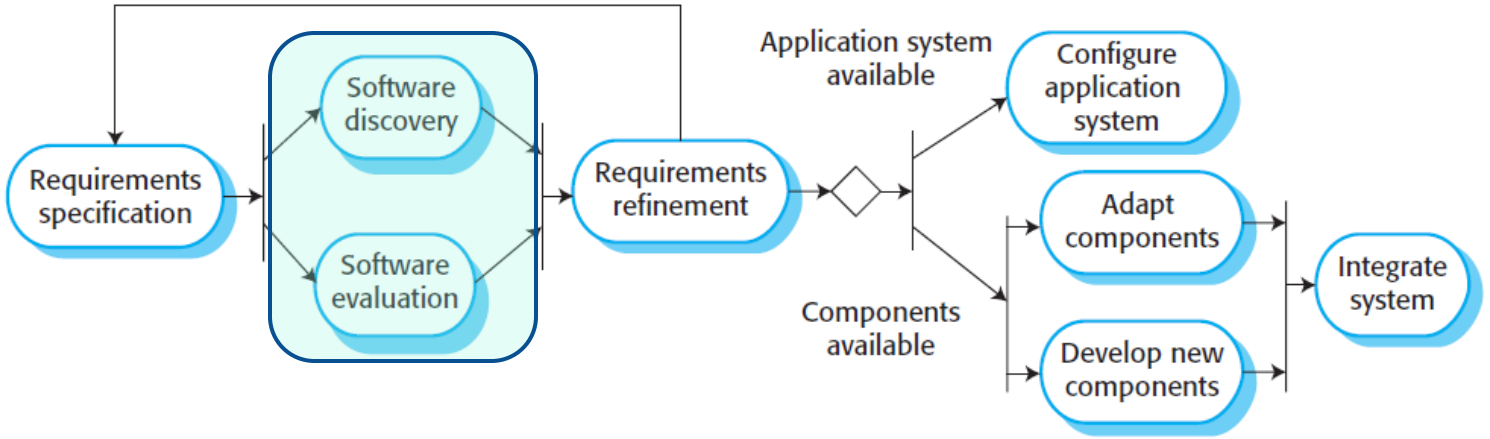


Reuse Oriented Software Engineering



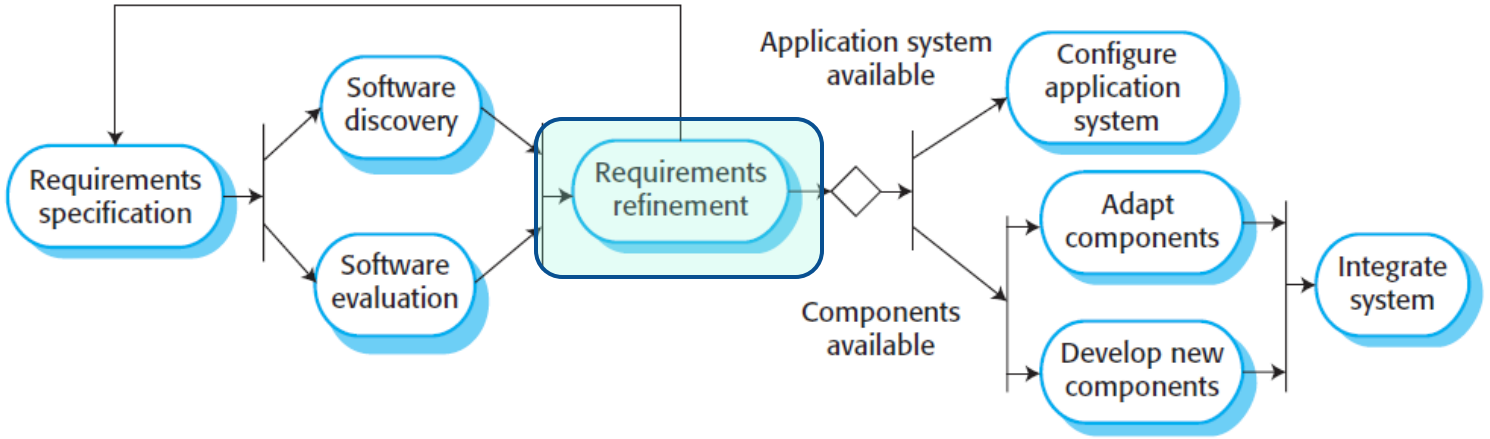
- Requirement specification;
  - initial requirements for the system are proposed.
  - These do not have to be elaborated in detail but should include brief descriptions of essential requirements and desirable system features.

Reuse Oriented Software Engineering



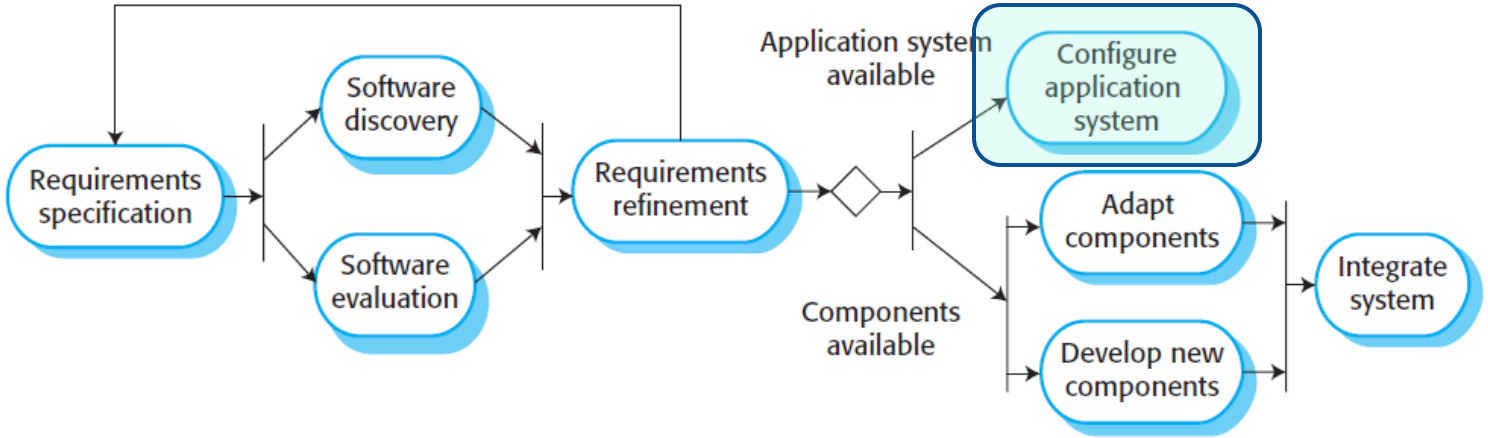
- Software Discovery and Evaluation
  - Given an outline of the software requirements, a search is made for components and systems that provide the functionality required.
  - Candidate components and systems are evaluated to see if they meet the essential requirements and if they are generally suitable for use in the system

Reuse Oriented Software Engineering



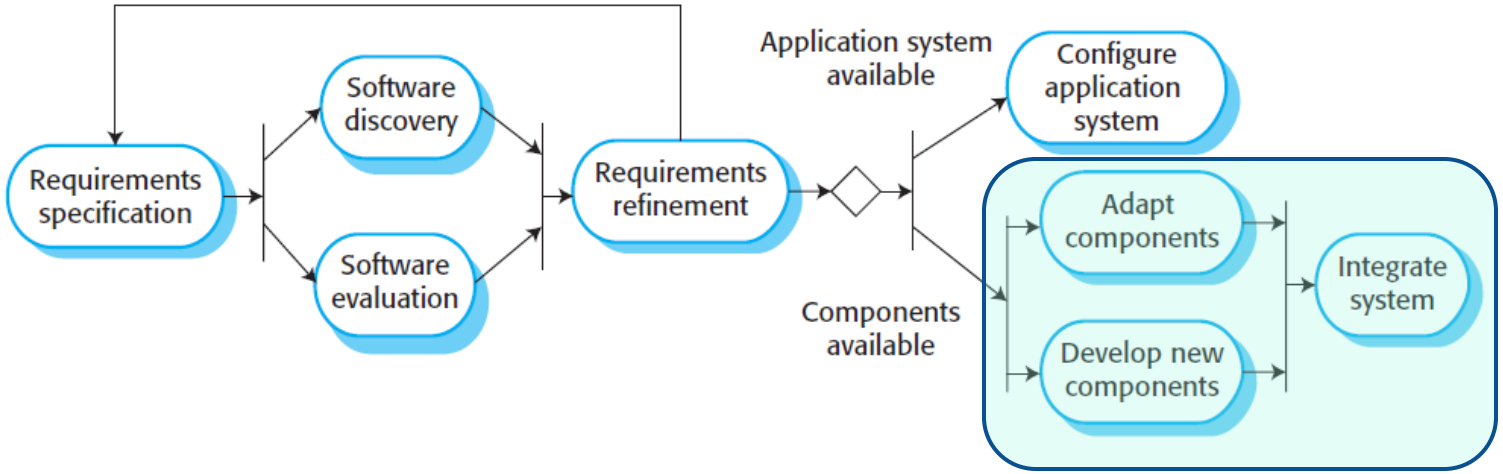
- Requirements refinement:
  - During this stage, the requirements are refined using information about the reusable components and applications that have been discovered.
  - The requirements are modified to reflect the available components, and the system specification is re-defined. Where modifications are impossible, the component analysis activity may be reentered to search for alternative solutions.

Reuse Oriented Software Engineering



- Application system configuration:
  - If an off-the-shelf application system that meets the requirements is available, it may then be configured for use to create the new system.

Reuse Oriented Software Engineering



- Component adaptation and integration:
  - If there is no off-the-shelf system, individual reusable components may be modified and new components developed.
  - These are then integrated to create the system.

Reuse Oriented Software Engineering

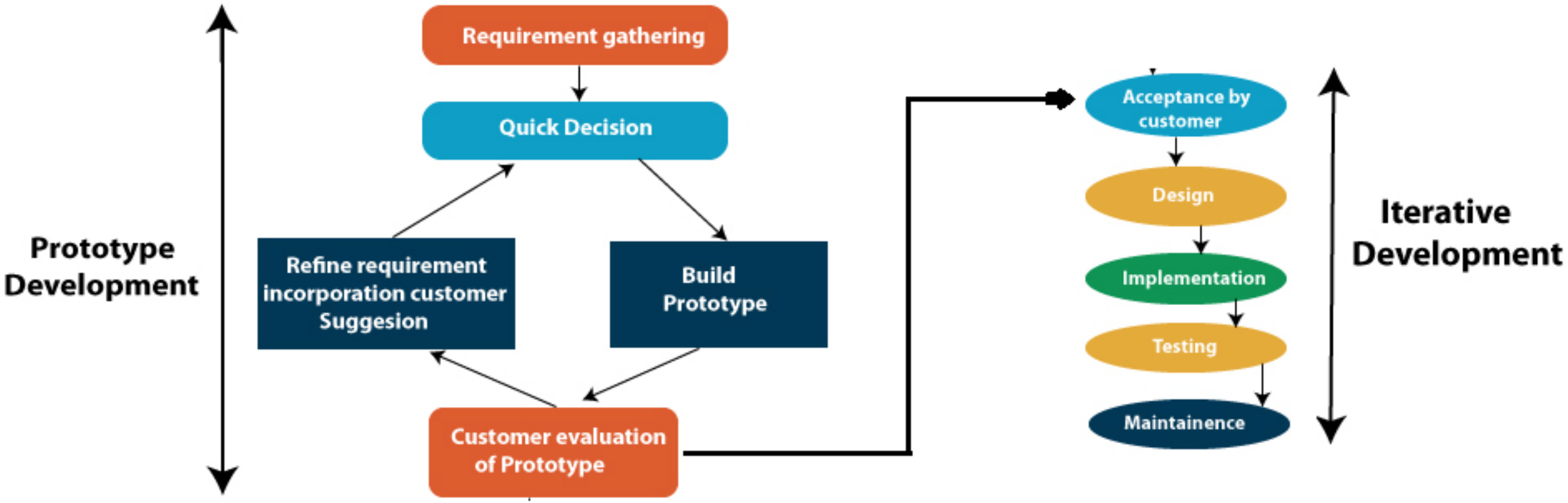
Advantages	Disadvantages
It can reduce total cost of software development.	Reuse-oriented model is not always worked as a practice in its true form.
The risk factor is very low.	Compromises in requirements may lead to a system that does not fulfill requirement of user.
It can save lots of time and effort.	Sometimes using old system component, that is not compatible with new version of component, this may lead to an impact on system evolution.
It is very efficient in nature.	

Prototyping Models

Prototyping Model

- A prototype is built, tested and then reworked as necessary until an acceptable outcome is achieved from which the complete system or product can be developed.
- This model works best in scenarios where not all of the project requirements are known in detail ahead of time.
- It is an iterative, trial-and-error process that takes place between the developers and the users.

Prototyping Model



Types of Prototyping Models

- Rapid Throwaway Prototype
  - Rapid throwaway is based on the preliminary requirement.
  - It is quickly developed to show how the requirement will look visually.
  - The customer's feedback helps drives changes to the requirement, and the prototype is again created until the requirement is baselined.
  - In this method, a developed prototype will be discarded and will not be a part of the ultimately accepted prototype.
  - This technique is useful for exploring ideas and getting instant feedback for customer requirements.

Types of Prototyping Models

- Evolutionary Prototyping
  - Here, the prototype developed is incrementally refined based on customer's feedback until it is finally accepted.
  - It helps to save time as well as effort. That's because developing a prototype from scratch for every interaction of the process can sometimes be very frustrating.
  - This model is helpful for a project which uses a new technology that is not well understood.
  - It is also used for a complex project where every functionality must be checked once.
  - It is helpful when the requirement is not stable or not understood clearly at the initial stage.

Types of Prototyping Models

- Incremental Prototyping
  - In incremental Prototyping, the final product is decimated into different small prototypes and developed individually.
  - Eventually, the different prototypes are merged into a single product.
  - This method is helpful to reduce the feedback time between the user and the application development team.

Types of Prototyping Models

- Extreme Prototyping:
  - Extreme prototyping method is mostly used for web development. It is consists of three sequential phases.
  - Basic prototype with all the existing page is present in the HTML format.
  - You can simulate data process using a prototype services layer.
  - The services are implemented and integrated into the final prototype.

Prototype Model

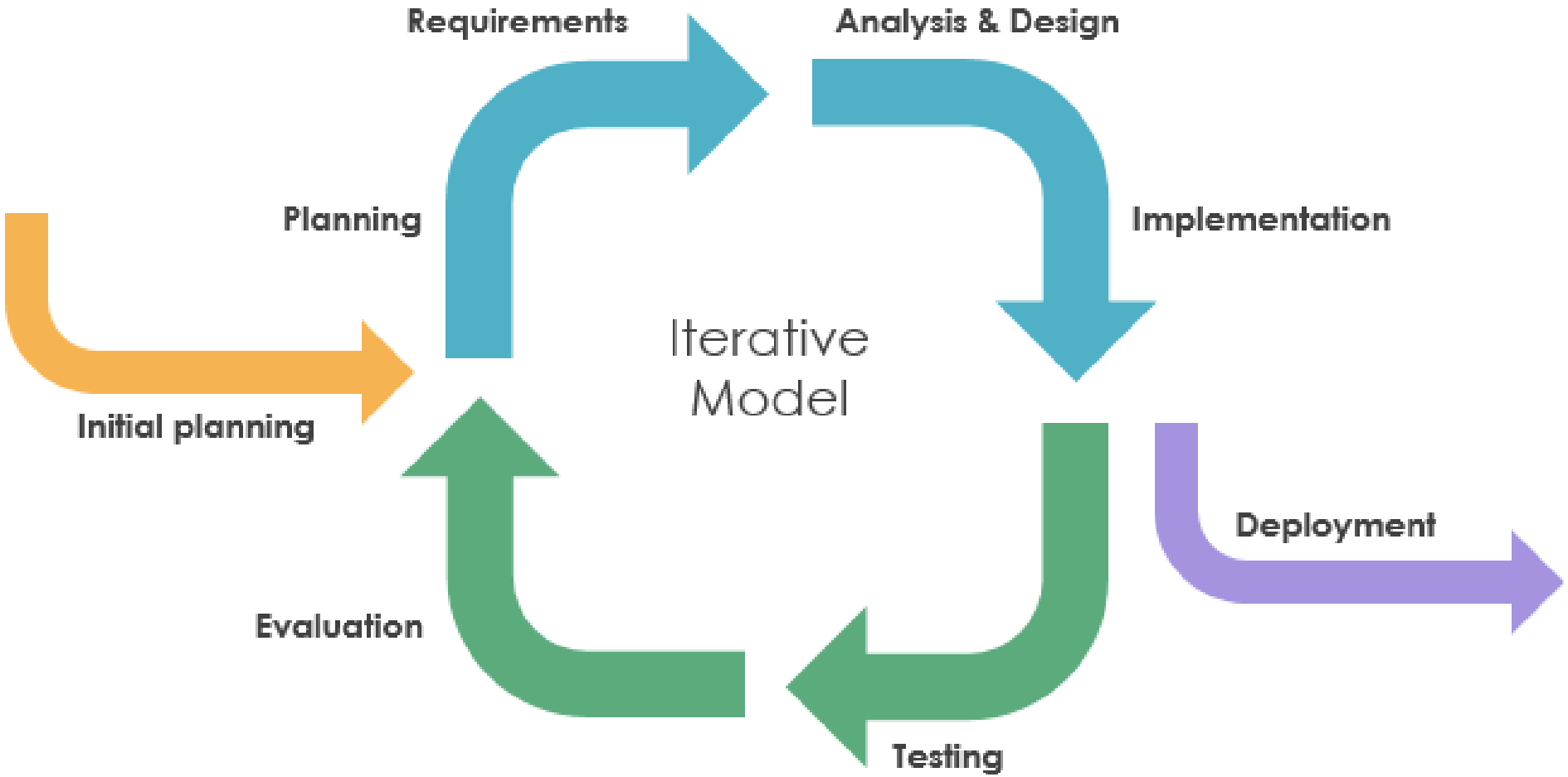
Advantages	Disadvantages
Errors can be detected in the initial stage of the software development process.	Prototyping is a slow and time taking process.
Missing functionality can be identified, which helps to reduce the risk of failure as Prototyping is also considered as a risk reduction activity	The cost of developing a prototype is a total waste as the prototype is ultimately thrown away.
Helps team member to communicate effectively	Prototyping may encourage excessive change requests.
Customer satisfaction exists because the customer can feel the product at a very early stage.	Poor documentation because the requirements of the customers are changing.

Iterative Development Model

Iterative Model

- Iterative software process models begin with a smaller set of requirements.
- Developers use these requirements to analyze and gradually implement features.
- Afterward, these features are evaluated and tested to identify new requirements.
- This cycle is called an “iteration.”
- This life cycle repeats numerous times, introducing new design choices, coding, and requirements changes.

Iterative Model



Iterative Model

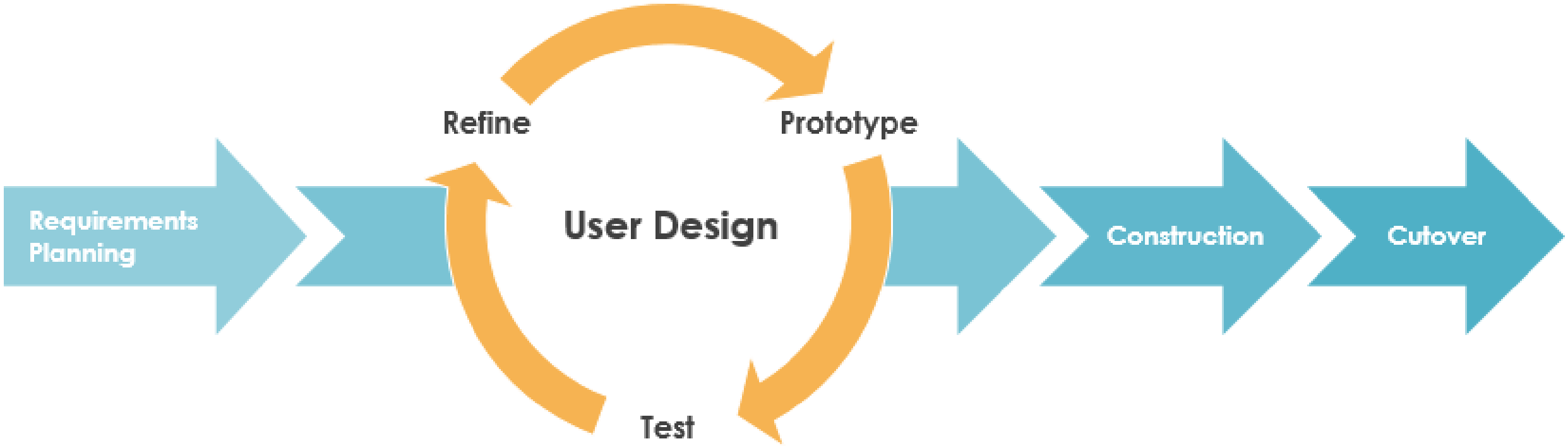
Advantages	Disadvantages
Coding begins early on	Difficult to analyze risks
A cost-effective way to implement requirement changes	Potential design and architecture issues in the later phases
Streamlined management because the production is divided into smaller pieces (iterations)	Too reliant on the baseline plan
Bugs and defects are easy to spot at earlier SDLC phases	A resource-heavy model

Rapid Application Development (RAD)

RAD Model

- Rapid Application Development (RAD) is an iterative software development process model that emphasizes an extremely short development cycle.
- If requirements are well understood and project scope is constrained, the RAD process enables a development team to create a 'fully functional system' within very short time periods (eg. 60 to 90 days)

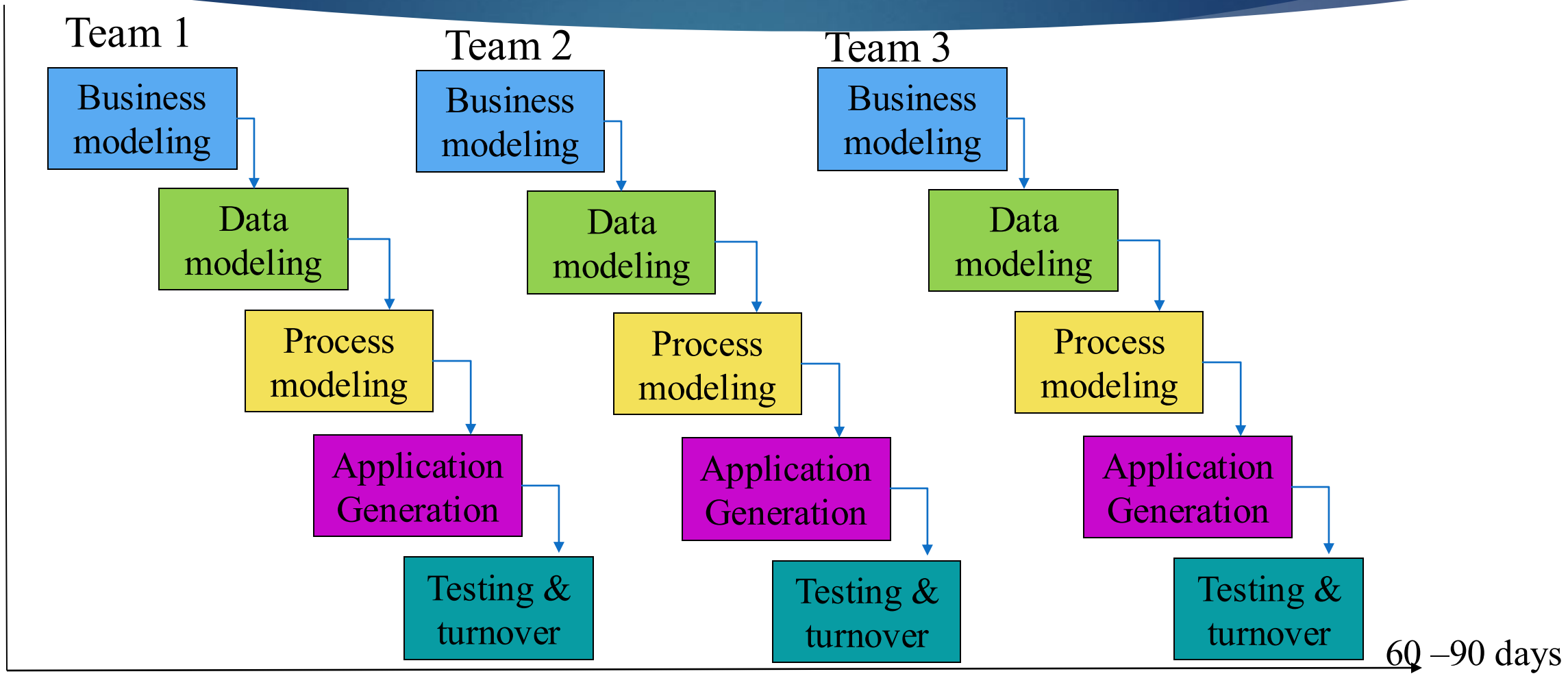
RAD Model


Refine
Prototype
Requirements Planning
User Design
Construction
Cutover
Test

Processes in the RAD Model

- **Business modelling -** The information flow in a business system considering its functionality.
- **Data Modelling -** The information flow defined as part of the business modelling phase is refined into a set of data objects that are needed to support the business
- **Process Modelling -** The data objects defined in the data modelling phase are transformed to achieve the information flow necessary to implement business functions.
- **Application generation -** RAD assumes the use of 4GL or visual tools to generate the system using reusable components.
- **Testing and turnover -** New components must be tested and all interfaces must be fully exercised

Processes in the RAD Model



Issues in RAD Model

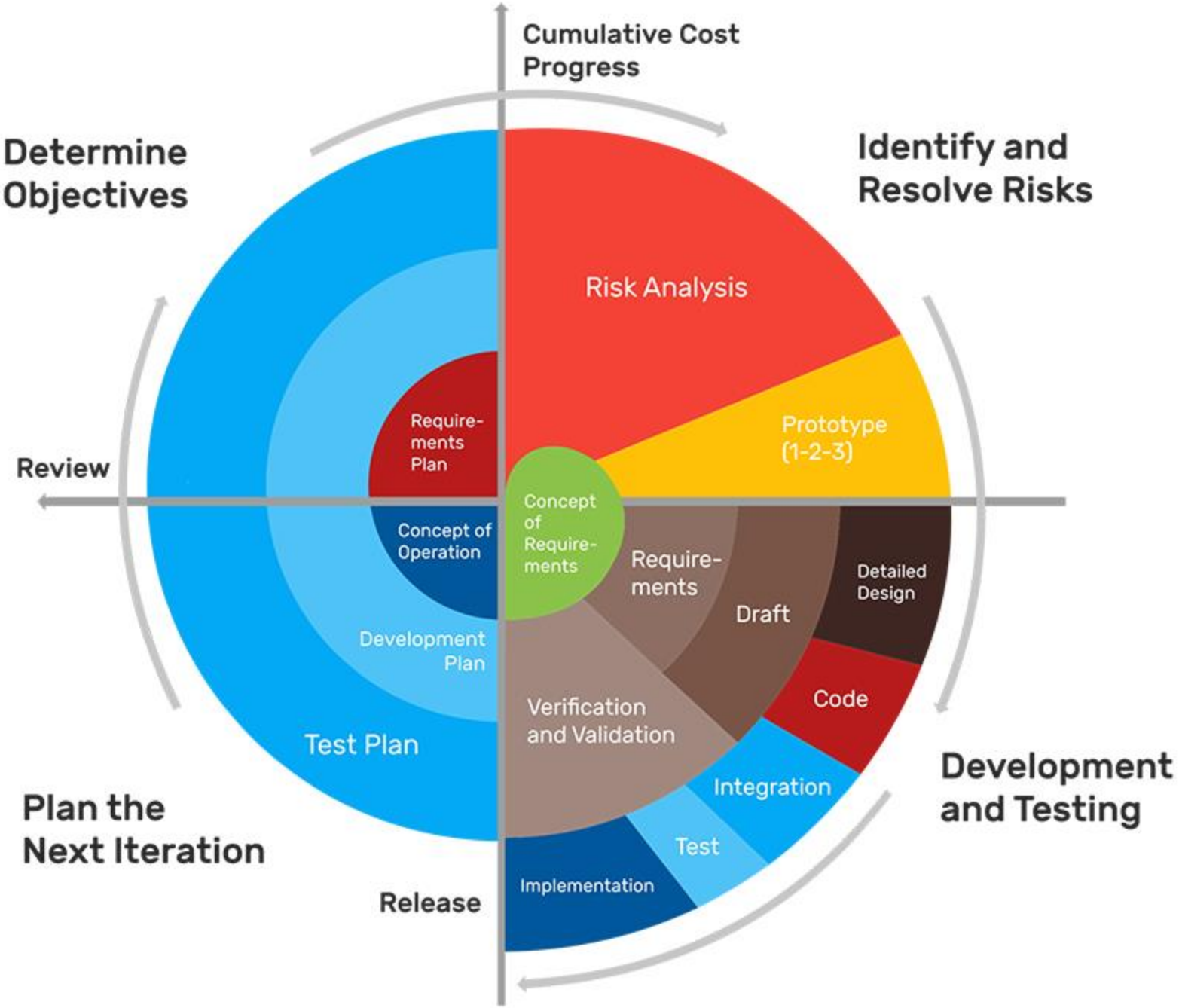
- RAD requires sufficient human resources to create right number of RAD teams
- RAD requires developers and customers who are committed to the rapid-fire activities necessary to get a system completed in a much abbreviated time frame.
- If a system cannot be properly modularized, building the components necessary for RAD will be problematic.
- RAD is not applicable when technical risks are high. This occurs when a new application makes heavy use of new technology or when the new software requires a high degree of interoperability with existing computer programs.

Spiral Model

Spiral Model

- Spiral combines the **Iterative and Waterfall methodologies** with an emphasis on risk assessment.
- It consists of the usual SDLC phases built on the baseline spiral.
- The spiral model delivers projects in loops.
- Each phase addresses whatever problem has the greatest risk of causing a failure.
- The loop repeats until the product complies with requirements.

Spiral Model



Spiral Model

Advantages	Disadvantages
Precisely documented	Success depends on skilled risk managers
Accurate time and budget estimates	Requires a large resource pool
Excellent risk assessment	Time-consuming
Engineers can apply changes to new iterations	

Process Models

Model	Suitable for:
Waterfall	• Smaller and mid-sized projects with clearly defined requirements
Incremental	• Projects that have loosely-coupled parts and projects with complete and clear requirements.
Iterative	• Large-scale projects with multiple modules (such as web services or microservices) • Projects with clearly defined objectives and tasks
RAD	• Systems that need to be produced in a short time and have known requirements.
Spiral	• Larger projects with complex requirements • New products with multiple testing stages

Group Assignment 02

- Presentation Duration: 10-15 minutes
- Each member should contribute to prepare the presentation (include a slide with individual contribution – list this in the same order you will be presenting at the class)
- Each member should present

Task: Compare and Contrast Plan-Driven Software Process Models

You are required to prepare and deliver a presentation that **compares and contrasts major plan-driven software process models** used in software engineering. Your presentation should explore the **structure**, **strengths**, **weaknesses**, and **use cases** of each model, with practical examples.

Include these aspects in your comparison:

- **Process structure and phases**
- **Planning approach**
- **Risk management**
- **Documentation and control**
- **Customer involvement**
- **Best-fit project types**

Tips:

- Showcase the principles and workflows of plan-driven software models.
- Analise similarities and differences between models.
- Evaluate the suitability of each model in different software project contexts.
- Practice effective technical communication and presentation skills.